

```

// A safe generic public YouTube video (embedding
allowed) used to demonstrate playback.
const DEMO_YT_ID = 'M7lc1UVf-VE';

let state = { cls:null, subjIdx:null, chapIdx:null,
videoIdx:null };

// ----- REAL PROGRESS TRACKING (saved
on this device, no login needed) -----
function progressKey(cls, subjIdx, chapIdx,
videoIdx){
    return
    `shikhoni:${cls}:${subjIdx}:${chapIdx}:${videoIdx}`
    ;
}
function getProgress(cls, subjIdx, chapIdx,
videoIdx){
    try{
        const v = localStorage.getItem(progressKey(cls,
subjIdx, chapIdx, videoIdx));
        return v ? parseFloat(v) : 0;
    }catch(e){ return 0; }
}
function setProgress(cls, subjIdx, chapIdx,
videoIdx, fraction){
    try{
        const current = getProgress(cls, subjIdx,
chapIdx, videoIdx);
        if(fraction > current){
            localStorage.setItem(progressKey(cls,
subjIdx, chapIdx, videoIdx),
String(Math.min(fraction,1)));
        }
    }catch(e){}
}

```

```

function isDone(cls, subjIdx, chapIdx, videoIdx){
    return getProgress(cls, subjIdx, chapIdx,
videoIdx) >= 0.9;
}

// Load the YouTube IFrame API so we can detect
when a video refuses to embed.
function setCrumb(parts){
    const el = document.getElementById('crumb');
    el.innerHTML = parts.map((p,i)=>{
        const isLast = i===parts.length-1;
        return (i>0?'<span class="sep">></span>':'' ) +
(isLast? `<span>${p.label}</span>` : `<button
onclick="${p.action}">${p.label}</button>`);
    }).join('');
}

function showView(id){

document.querySelectorAll('.view').forEach(v=>v.class
ssList.remove('active'));

document.getElementById(id).classList.add('active')
;
    window.scrollTo({top:0, behavior:'smooth'});
}

let navigatingFromHistory = false;

function goHome(){
    state =
{cls:null,subjIdx:null,chapIdx:null,videoIdx:null};
    const picker = document.getElementById('class-
picker');
    picker.innerHTML = Object.keys(DATA).map(key => {
        const g = DATA[key];
        return `<div class="class-card"
onclick="goSubjects('${key}')">
        <div class="tag">${g.tag}</div>
        <h3>${g.title}</h3>

```

```

        <div class="desc">${g.desc}</div>
        <div class="go">সুরু করো →</div>
    </div>`;
    }).join('');
    setCrumb([{label:'শিখনী', action:'goHome()'}]);
    showView('view-home');
    if(!navigatingFromHistory){
    history.pushState({view:'home', '', ''}); }
    }

function goSubjects(cls){
    state.cls = cls; state.subjIdx=null;
    state.chapIdx=null;
    document.getElementById('subj-
    eyebrow').textContent = `${DATA[cls].tag} ·
    ${DATA[cls].title}`;
    const grid = document.getElementById('subject-
    grid');
    grid.innerHTML = DATA[cls].subjects.map((s,i)=>{
        const chCount = s.chapters.length;
        return `<div class="subject-card"
    onclick="goChapters(${i})">
        <div class="icon">${s.icon}</div>
        <h3>${s.name}</h3>
        <div class="count">${chCount} অধ্যায়</div>
    </div>`;
    }).join('');
    setCrumb([{label:'শিখনী', action:'goHome()'},
    {label:DATA[cls].title,
    action:`goSubjects('${cls}')`}]);
    showView('view-subjects');
    if(!navigatingFromHistory){
    history.pushState({view:'subjects', cls}, '', '');
    }
    }

function goChapters(subjIdx){
    state.subjIdx = subjIdx;
    const subj = DATA[state.cls].subjects[subjIdx];
    document.getElementById('chap-

```

```

eyebrow').textContent = `${state.cls} ·
${subj.name}`;
    const toc = document.getElementById('chapter-
toc');
    toc.innerHTML = subj.chapters.map((c,i)=>{
        const dots = c.classes.map((x,j)=>`<i
class="${isDone(state.cls, subjIdx, i,
j)}?'done':'}'"></i>`).join('');
        return `<div class="toc-item"
onclick="goClasses(${i})">
        <span class="toc-num
mono">${String(i+1).padStart(2,'0')}</span>
        <span class="toc-title">${c.name}</span>
        <span class="toc-dots"></span>
        <span class="toc-meta">${c.classes.length}
        ক্লাস</span>
        <span class="progress-dots">${dots}</span>
        </div>`;
    }).join('');
    setCrumb([
        {label:'শিখনী', action:'goHome()'},
        {label:DATA[state.cls].title,
        action:`goSubjects('${state.cls}')`},
        {label:subj.name,
        action:`goChapters(${subjIdx})`}]
    );
    showView('view-chapters');
    if(!navigatingFromHistory){
        history.pushState({view:'chapters', cls:state.cls,
        subjIdx, '', ''});
    }
}

function goClasses(chapIdx){
    state.chapIdx = chapIdx;
    const subj =
DATA[state.cls].subjects[state.subjIdx];
    const chap = subj.chapters[chapIdx];
    document.getElementById('class-
eyebrow').textContent = `${state.cls} ·
${subj.name}`;
    document.getElementById('class-
heading').textContent = chap.name;

```

```

    renderClassList('class-list', chap.classes,
true);
    setCrumb([
{label:'শিখনী', action:'goHome()'},
{label:DATA[state.cls].title,
action:`goSubjects('${state.cls}')`},
{label:subj.name,
action:`goChapters('${state.subjIdx}')`},
{label:chap.name,
action:`goClasses('${chapIdx}')`}}]);
    showView('view-classes');
    if(!navigatingFromHistory){
history.pushState({view:'classes', cls:state.cls,
subjIdx:state.subjIdx, chapIdx, '', ''}); }
}

function renderClassList(elId, classes, clickable){
    const el = document.getElementById(elId);
    el.innerHTML = classes.map((c,i)=>{
        const prog = getProgress(state.cls,
state.subjIdx, state.chapIdx, i);
        const done = prog >= 0.9;
        const pct = Math.round(prog*100);
        return `
        <div class="class-row" ${clickable?
`onclick="playVideo(${i})"``:'`}>
        <div class="thumb">
            <span class="play">▶</span>
            ${pct>0 ? `<div class="thumb-fill"
style="width:${pct}%"></div>` : ''}
        </div>
        <div class="meta">
            <div class="n">ক্লাস ${i+1} · ${c.n}</div>
            <div class="d mono">${c.dur}</div>
        </div>
        <div class="status">${done ? '✅' : (pct>0 ?
`<span class="pct mono">${pct}%</span>` : '')}
        </div>
        </div>`;
    }).join('');
}

```

```

function playVideo(videoIdx){
  state.videoIdx = videoIdx;
  const subj =
DATA[state.cls].subjects[state.subjIdx];
  const chap = subj.chapters[state.chapIdx];
  const video = chap.classes[videoIdx];

  document.getElementById('p-title').textContent =
`ক্লাস ${videoIdx+1} · ${video.n}`;
  document.getElementById('p-duration').textContent
= video.dur;
  document.getElementById('p-chapter-
name').textContent = chap.name;
  const ytId = video.yt || DEMO_YT_ID;
  loadYoutube(ytId);

  const noteCard = document.getElementById('note-
card');
  if(video.note){
    noteCard.style.display = 'flex';
    document.getElementById('note-link').href =
video.note;
  } else {
    noteCard.style.display = 'none';
  }

  const hasNext = videoIdx+1 < chap.classes.length;
  document.getElementById('up-next').style.display
= hasNext ? 'flex' : 'none';
  if(hasNext){
    document.getElementById('next-
title').textContent = `ক্লাস ${videoIdx+2} ·
${chap.classes[videoIdx+1].n}`;
  }
  renderClassList('sibling-list', chap.classes,
false);
  document.querySelectorAll('#sibling-list .class-
row').forEach((row,i)=>{
    row.style.cursor='pointer';

```

```

        row.onclick=(()=>playVideo(i);
        if(i===videoIdx) row.style.borderColor='var(--
gold)';
        });

        setCrumb([
            {label:'শিখনী', action:'goHome()'},
            {label:DATA[state.cls].title,
            action:`goSubjects('${state.cls}')`},
            {label:subj.name,
            action:`goChapters(${state.subjIdx})`},
            {label:chap.name,
            action:`goClasses(${state.chapIdx})`}, {label:`ক্লাস
${videoIdx+1}`, action:''}]);
        showView('view-player');
        if(!navigatingFromHistory){
            history.pushState({view:'player',
            cls:state.cls, subjIdx:state.subjIdx,
            chapIdx:state.chapIdx, videoIdx}, '', '');
        }
    }

    function playNext(){
        playVideo(state.videoIdx+1);
    }

    let ytPlayer = null;
    let ytApiReady = false;
    let progressTimer = null;

    const ytScript = document.createElement('script');
    ytScript.src =
    'https://www.youtube.com/iframe_api';
    document.head.appendChild(ytScript);
    window.onYouTubeIframeAPIReady = function(){
        ytApiReady = true; };

    function loadYoutube(videoId){
        document.getElementById('ff-link-2').href =
        `https://www.youtube.com/watch?v=${videoId}`;
        document.getElementById('frame-

```

```

error').classList.remove('show');
    document.getElementById('p-progress-
fill').style.width = '0%';
    clearInterval(progressTimer);

    const create = () => {
        if(ytPlayer){
            ytPlayer.loadVideoById(videoId);
        } else {
            ytPlayer = new YT.Player('p-frame', {
                width: '100%',
                height: '100%',
                videoId: videoId,
                playerVars: { rel:0, modestbranding:1,
fs:1, playsinline:1 },
                events: {
                    'onReady': () => {},
                    'onStateChange': onPlayerStateChange,
                    'onError': (e) => {
                        // 101/150 = owner disabled embedding,
100 = video removed/private
                        document.getElementById('frame-
error').classList.add('show');
                        clearInterval(progressTimer);
                    }
                }
            });
        }
    };

    if(ytApiReady){ create(); } else {
        window.onYouTubeIframeAPIReady = function(){
ytApiReady = true; create(); };
    }
}

function onPlayerStateChange(e){
    clearInterval(progressTimer);
    if(e.data === YT.PlayerState.PLAYING){
        progressTimer = setInterval(() => {

```



```

        if(!ytPlayer || !ytPlayer.getDuration)
        return;
        const dur = ytPlayer.getDuration();
        const cur = ytPlayer.getCurrentTime();
        if(dur > 0){
            const frac = cur/dur;
            document.getElementById('p-progress-
            fill').style.width = `${frac*100}%`;
            setProgress(state.cls, state.subjIdx,
            state.chapIdx, state.videoIdx, frac);
        }
    }, 1000);
}
if(e.data === YT.PlayerState.ENDED){
    setProgress(state.cls, state.subjIdx,
    state.chapIdx, state.videoIdx, 1);
    document.getElementById('p-progress-
    fill').style.width = '100%';
}
}

navigatingFromHistory = true;
goHome();
navigatingFromHistory = false;
history.replaceState({view:'home'}, '', '');

window.addEventListener('popstate', function(e){
    navigatingFromHistory = true;
    const s = e.state;
    if(!s || s.view === 'home'){
        goHome();
    } else if(s.view === 'subjects'){
        goSubjects(s.cls);
    } else if(s.view === 'chapters'){
        state.cls = s.cls;
        goChapters(s.subjIdx);
    } else if(s.view === 'classes'){
        state.cls = s.cls; state.subjIdx = s.subjIdx;
        goClasses(s.chapIdx);
    } else if(s.view === 'player'){

```

```

        state.cls = s.cls; state.subjIdx = s.subjIdx;
state.chapIdx = s.chapIdx;
        playVideo(s.videoIdx);
    } else {
        goHome();
    }
    navigatingFromHistory = false;
});

if('serviceWorker' in navigator){
    window.addEventListener('load', () => {

navigator.serviceWorker.register('sw.js').catch(()=>{});
    });
}

// ----- CUSTOM FULLSCREEN (works better
// than YouTube's own button inside app-wrapped APKs)
// -----
function toggleFullscreen(){
    const box = document.getElementById('frame-box');
    const isFs = document.fullscreenElement ||
document.webkitFullscreenElement;

    if(!isFs){
        const req = box.requestFullscreen ||
box.webkitRequestFullscreen ||
box.webkitEnterFullscreen;
        if(req){
            req.call(box).then(() => {
                if(screen.orientation &&
screen.orientation.lock){

screen.orientation.lock('landscape').catch(()=>{});
                }
            }).catch(() => {
                alert('এই ডিভাইস/অ্যাপে ফুলস্ক্রিন সাপোর্ট নেই। ফোনটা
পাশ (landscape) করে ধরলে ডিডিও এমনিতেই বড় দেখাবে!');
            });
        }
    }
}

```

```
    } else {  
        alert('এই ডিভাইস/অ্যাপে ফুলস্ক্রিন সাপোর্ট নেই। ফোনটা  
পাশ (landscape) করে ধরলে ভিডিও এমনতেই বড় দেখাবে।');  
    }  
    } else {  
        const exit = document.exitFullscreen ||  
document.webkitExitFullscreen;  
        if(exit) exit.call(document);  
        if(screen.orientation &&  
screen.orientation.unlock){  
            screen.orientation.unlock();  
        }  
    }  
}
```